Transferring and Reading Skills in Language Model Contexts

Anonymous Author(s)

ABSTRACT

Skills supplied in a language model's context can improve task performance, but how the model represents and uses their content remains unclear. Can one hidden-state vector carry a skill's effect, or are states at multiple token positions needed? We compare a correct skill with a wrong skill of the same token length, then inject the difference between their hidden states. In synthetic tasks, the final prompt position carries a large change shared by both skills, yet the content-specific difference accounts for most of the behavioural gain. Recovery at this position depends on answer format: it falls from 1.000 for a one-token answer to 0.235 for a short number and 0.003 for step-by-step reasoning. Injecting states across the skill's own token span instead recovers 0.89 of its gain on a screened clinical-calculator subset of 135 items. Recovery is sensitive to skill content and the placement of injected states. Keeping only the leading 32 directions of the centred state difference recovers 0.12, compared with 0.77 for 128 directions. The rank needed to reach half of the measured recovery range does not scale proportionally with model width within Qwen. Mistral's is higher, though with overlapping intervals. Across two task domains and two model families, paired interventions further show that span states stop transferring most of the skill's effect before the model stops depending on attention to them. These findings identify a transferable representation across skill tokens and clarify the limits of probing it at a single prompt position.

CCS CONCEPTS

• **Computing methodologies** → **Natural language processing**; *Knowledge representation and reasoning*.

KEYWORDS

large language models, skill documents, in-context learning, activation patching, representation rank, mechanistic interpretability

ACM Reference Format:

Anonymous Author(s). 2027. Transferring and Reading Skills in Language Model Contexts. In *Proceedings of Proceedings of the ACM Web Conference 2027 (WWW '27).* ACM, New York, NY, USA, 12 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

A skill is a reusable procedure, formula, lookup table or worked example supplied in a language model's context to help it solve a task. Recent benchmarks show that skills can improve performance [20, 33], but higher accuracy alone does not explain how a model uses them. Does a single hidden-state vector carry the useful content, or does the model rely on states distributed across the skill's token positions? How long does it continue to read those states? We address these questions by intervening on the model's representations and measuring how much of the skill's performance gain they recover. Prior work transfers task information from demonstrations and instructions through compact states [3, 6, 15, 36], but transfer can depend on token position and task scope [21]. We examine long procedural contexts, pairing state transfer with interventions on continued access to the source tokens.

Our starting point is a contrast between behaviour and hidden-state geometry. Across 1,100 clinical-calculator instances, the correct skill improves accuracy by 39.7 percentage points over no skill, whereas a wrong skill of the same token length changes it by −2.9 points (95% interval $[-8.6, +2.9]$, paired calculator bootstrap). Yet in a synthetic task, correct and wrong skills produce similar changes at the final prompt position: their state differences relative to no skill have cosine similarity 0.91–0.95 over layers 0–10. These observations motivate a causal question: does the shared state change support the improvement, or is the content-specific difference responsible?

We subtract hidden states and inject each difference back into the model. On the synthetic task, the change from no skill to a wrong skill recovers only 0.098 of the correct skill's gain, whereas the correct-minus-wrong difference recovers 0.805. Recovery also depends on answer format: the best state replacement recovers 1.000 for one-token multiple choice, 0.235 for a short number and 0.003 for step-by-step reasoning. Success at the final prompt position can therefore reflect transfer of a compact answer decision, while failure need not imply that the skill lacks a transferable representation.

We find that *states across the skill's own token span can transfer much of its effect.* Replacing a wrong skill's states with the correct skill's at one layer recovers 0.89 of its gain on a screened subset of 135 items from 14 clinical calculators. Halving the injection, permuting its positions or substituting norm-matched noise recovers at most 0.05. Truncating the matrix of state differences tests how many directions support recovery. With the positional mean preserved, 32 leading directions recover 0.12, 128 recover 0.77, and the bottom 64 recover only 0.03. Both position and the retained directions matter for this intervention.

We compare models by the number of directions needed to reach half of the measured recovery range. Reducing hidden width fourfold within Qwen changes this value from 47 to 43, rather than the pre-registered proportional prediction of approximately 12. Mistral at the larger width gives 64–72, with intervals that overlap Qwen's. Across layers, skill states stop transferring most of the effect while attention to them still supports performance. This separation replicates wherever both interventions are available. A few unusually large activations explain an apparent drop

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
WWW '27, May 10–14, 2027, Dublin, Ireland
© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-X-XXXX-XXXX-X/27/04…$15.00
https://doi.org/XXXXXXX.XXXXXXX

in dimensionality, which therefore does not establish compression. Our contributions are:

(1) **Locating a transferable skill representation.** Matched-control subtraction isolates content-specific state differences, answer-format comparisons reveal the limits of a single-position probe, and span injection identifies a representation that recovers the skill's effect (§4.2–4.5).
(2) **Measuring the representation rank required for recovery.** Truncating the injected matrix shows that recovery depends on multiple leading directions. The measured rank curves depart from proportional width scaling within Qwen (§4.6, §4.8).
(3) **Separating skill transfer from continued reading.** Span replacement and blocking attention to skill tokens reveal distinct depth boundaries. Multi-layer interventions test whether repeated late replacement restores transfer. Spectral checks identify the influence of unusually large activations (§4.7).

2 RELATED WORK

Skills and compact context representations. Skill benchmarks ask whether agents retrieve and apply reusable procedures [12, 20, 33]. We ask how a model represents one already in context. Task and function vectors transfer behaviours from in-context demonstrations through compact activations [15, 36], and task arithmetic makes related edits to weights [16]. Instruction-derived function vectors can involve components distinct from those used by demonstrations [6]. Transferable task representations also have temporal and semantic locality [21], including variation across extended outputs. Bigoulaeva et al. [3] patch the final instruction token before the query and find superadditive effects across layers. Dong et al. [7] analyse single-vector limitations in a linear-attention setting and test multiple task vectors. Their task-mapping rank differs from our rank of state differences across procedural tokens. Learned context compression uses aggregated activations [1], summary tokens [26], compact states [10], weight updates [4, 5]. We examine the original model's states without training a compression mechanism.

Answer representations and activation interventions. Multiple-choice tasks require selecting an answer symbol [38], whereas reasoning can depend on states distributed over generated positions [8]. Transferring states from completed reasoning [24] therefore tests a different setting from ours, which captures states before generation begins. Repeatedly applying activation directions can also steer extended generations [22, 27, 32]. Our single-position tests instead use one-time residual-state patches before generation, a different protocol from head-selected function vectors. Following activation-patching methodology [14, 40], we compare generated answers against explicit baselines and verify that writing a run's own states back leaves it unchanged. Prior work traces factual information across layers [11, 25]. Sia et al. [31] use cumulative context masking, while Pola and Balasubramanian [28] compare instruction-span and final-position patches. We pair span transfer with cumulative attention knockout in the same setting to distinguish transferability from continued reading dependence.

Representation rank and spectral measurements. Spectral analyses use relative singular-value sizes to study hidden-state geometry [37], sensitivity to perturbations [29], and transformations across layers [9]. We truncate the matrix of content-specific state differences across skill positions and measure the resulting performance. Reports of unusually large activations [30, 34] and dominant hidden dimensions [19, 35] motivate a separate check: does an apparent drop in dimensionality reflect a change in useful content or merely a few large coordinates? We keep that geometric diagnostic distinct from the rank intervention's behavioural outcome.

3 PROBLEM FORMULATION

Skills, positions and representations. Let x be a question, s its correct skill, and $\tilde{s}$ an unrelated skill with the same token length. For a fixed autoregressive model of hidden width D, let $h^L_{\text{skill}}(p)$, $h^L_{\text{wrong}}(p)$ and $h^L_{\text{no}}(p)$ denote residual-stream hidden states at layer L and prompt position p with s, with $\tilde{s}$ and without a skill. At the final prompt position, we decompose the total displacement as

$$\underbrace{h_{\text{skill}} - h_{\text{no}}}_{t\ \text{(total)}} = \underbrace{h_{\text{wrong}} - h_{\text{no}}}_{g\ \text{(presence)}} + \underbrace{h_{\text{skill}} - h_{\text{wrong}}}_{d\ \text{(content)}}. \tag{1}$$

We omit layer and position indices in Eq. 1 for readability. "Presence" denotes the change induced by the chosen wrong skill. "Content" denotes the additional change induced by replacing it with the correct skill. This algebraic decomposition does not assume independent underlying factors. We test each component by adding it to the corresponding state in a no-skill run.

Across the skill's m token positions S, the content-specific difference is a matrix $d^L_S = H^L_{\text{skill}}(S) - H^L_{\text{wrong}}(S) \in \mathbb{R}^{m\times D}$. We use a run containing the wrong skill as the *receiver*, and the correct-skill run as the *donor*. Injection at dose α sets

$$H^L_{\text{patched}}(S) = H^L_{\text{wrong}}(S) + \alpha d^L_S. \tag{2}$$

At $\alpha = 1$, the donor's states replace the receiver's across the span. Other states remain unchanged at that layer. Computation then continues. Matching lengths preserves positions and their encodings. Span interventions change many more states than final-position interventions, which limits direct comparisons. Here the final position is the prompt's end, after the query. State patches are applied during prefill, after which decoding proceeds without further patches.

Behavioural recovery and evaluation subsets. We score generated answers and measure the fraction of the donor's accuracy gain recovered by an intervention:

$$\rho(\text{arm}) = \frac{\text{acc(arm)} - \text{acc(receiver)}}{\text{acc(donor)} - \text{acc(receiver)}}. \tag{3}$$

Here, an arm is an intervention condition, and the donor baseline uses the correct skill in context. Thus $\rho = 1$ recovers the donor's observed gain, and values above one indicate greater accuracy than the donor. We distinguish rescued items (R: wrong without the skill, correct with it), persistent failures (F: wrong under both), kept items (K: correct under both), and broken items (B: correct without, wrong with). Main causal comparisons use rescued items. For span interventions, we further retain only skill groups for which

the wrong-skill receiver solves none of the sampled items. We apply this screen separately within each run, model and task, without using intervention outcomes. The estimates are conditional on baseline failure in the sampled group, rather than recovery over the full benchmark.

Rank and depth as intervention variables. To test how many directions support recovery, we subtract the mean across positions, truncate the resulting matrix, and restore the mean:

$$d^{(k)} = \mathbf{1}\bar{d} + \sum_{i=1}^{k} \sigma_i u_i v_i^\top, \qquad d - \mathbf{1}\bar{d} = \sum_{i} \sigma_i u_i v_i^\top. \tag{4}$$

Here, $\bar{d} \in \mathbb{R}^{1\times D}$ is the mean state difference, repeated at every position. The singular values σ_i are ordered from largest to smallest, with u_i and v_i their associated directions. Thus k counts retained directions after centring, and the complete injected matrix has rank at most k+1. A bottom-k control keeps the k smallest singular directions and the same mean. We also vary the intervention layer. Span replacement tests whether states at that layer suffice to transfer the effect. Blocking attention to the span from that layer onward tests whether subsequent computation still depends on reading it.

4 EXPERIMENT

4.1 Experimental setup

Tasks and models. Our main evaluation pairs MedCalc-Bench [18] with skills from SRA-Bench [33], covering 55 clinical calculators with 20 questions each (1,100 items). Qwen3-8B [39] generates step-by-step reasoning using greedy decoding, without tools. We evaluate its answers with the benchmark scorer. Accuracy is 74.2% with the correct skill, 34.5% without a skill, and 31.5% with a wrong skill. For the decomposition and answer-format experiments, we use a synthetic unit-conversion task with a controlled lookup table and matched wrong skills. We also test Qwen3-0.6B, Qwen3-1.7B and Mistral-7B-Instruct-v0.3 [17], and evaluate transfer to SRA-Bench's TheoremQA subset (747 questions and 320 skills) using its prompt and scorer.

Sampling and uncertainty. The main MedCalc span experiments cover 467 rescued items from 48 calculators with at least two such items. Screening for groups whose receiver solves no sampled item retains 135 items from 14 calculators. The main depth sweep, rank curve and span-quarter comparisons use this same subset. Appendix B compares this rerun with the earlier development samples it replaces. TheoremQA uses the same group-based screen, which becomes an item-level screen when a skill has only one question. We report sample sizes for each comparison. Main-text accuracy and recovery intervals resample skill groups. Each recovery estimate uses donor and receiver baselines on the items available for that intervention. Incomplete reruns are excluded rather than combined into curves with changing sample sets. Complete earlier measurements are retained and identified.

Identity control. Writing the receiver's own states back should leave its output unchanged. This identity control exposed a mismatch: capture omitted an attention mask that decoding supplied, changing 108/1,766 Qwen3-8B outcomes (6.1%). Matching the masks restores exact agreement in every corrected run, for example on all 467 MedCalc items and on all 9,308 item–layer pairs of the synthetic decomposition. All main-text interventions use the correction. Appendix A.2 records provenance and Appendix A.1 implementation details. Small numerical discrepancies can propagate through long generations [13].

4.2 Resolving the paradox: content and answer format

The smaller content component carries the improvement. On the synthetic task with Qwen3-1.7B (358 items, 82 rescued), we inject each component of Eq. 1 at the final prompt position. On rescued items, the presence component recovers 0.098 of the gain, exactly the wrong-skill baseline. The content component recovers 0.805, compared with 1.000 for full state replacement. The effect extends to failures: across rescued, persistent-failure, kept and broken items, replacement gives accuracies of 1.000/0.000/1.000/0.000, while content injection gives 0.805/0.075/1.000/0.186. Content injection thus broadly reproduces both the benefits and the errors induced by the skill. Recovery also depends on the subtraction baseline. Using a skill with corrupted numbers or with shuffled sentences gives 0.17–0.28 over layers 21–27, every layer measured, compared with 0.805 for an unrelated skill. The content component must therefore be interpreted relative to the specified wrong skill.

Single-position recovery depends on answer format. Table 1 compares answer formats on the synthetic task. All three formats use the same 358 items and all 36 layers. For short numeric answers, no final-position intervention recovers more than one of the 119 rescued items through layer 22. The best results across layers are 0.235 for state replacement, 0.21 for content and 0.008 for presence. These answers are one to five tokens long (341 of 358 use two to four). Patching the last 1, 4, 16, 32, 48 and 61 positions (61 covers the whole no-document prompt) recovers 0.235, 0.261, 0.403, 0.403, 0.395 and 0.370: the ladder saturates at 16, and giving the patch every position the prompt has does not raise it. For step-by-step reasoning, the skill raises accuracy from 0.000 to 0.989, yet single-position replacement recovers at most one of 354 rescued items (0.003) at any layer. Recovery varies with answer format, which changes both output length and computation. Same-query replacement of the final-layer final state also copies the next-token decision. We therefore interpret the comparison as a test of this one-time intervention, alongside prior evidence that transfer depends on position and task scope [21].

The corrected clinical comparison tests the final position and larger sets of positions after the skill at layer 8, using the same 135 items and wrong-skill receiver as span injection (Figure 1). The last 1, 16 and up to 256 positions recover 0.03, 0.06 and 0.05, respectively. Earlier multi-layer probes are reported separately in the extended version. The complete matched comparison here concerns layer 8.

4.3 The skill span supports content injection

Changing the intervention positions restores recovery. We next apply Eq. 2 across the skill's own positions. At layer 8, span injection raises accuracy from 0.000 to 0.815 on the screened 135-item

Table 1: Final-position state replacement across answer formats, Qwen3-8B. All formats use the same 358 items and the same skill, with every layer (0–35) tested. Recovery is the best state-replacement result over layers on each format's rescued items. The identity intervention reproduces the unpatched output on every item and layer.

Answer format	Tokens	Evaluated	Rescued	Best recovery
Multiple choice	1	358	94	1.000
Numeric answer	1–5	358	119	0.235
Chain of thought	Hundreds	358	354	0.003

subset, recovering 0.89 of the donor's gain (Table 2). Figure 1 compares this with writing donor states at the final position, the final 16 positions, or up to 256 positions after the skill, using the same layer, items and receiver. All positions outside the chosen intervention retain their receiver states.

Dose, transfer, order and noise controls. Recovery changes nonlinearly with injection strength. On the screened subset ($n = 135$, 14 calculators), $\alpha = 0.4, 0.5, 0.55$ gives 0.04, 0.04, 0.14 recovery, while 0.6, 0.7, 1.0 gives 0.53, 0.87, 0.89. Doubling the injection to $\alpha = 2$ reduces recovery to 0.77. Most of the increase occurs between 0.55 and 0.7. To test content specificity, we compare correct and other-skill injections over a common prefix length: the correct prefix recovers 0.65, whereas content from an unrelated skill recovers 0.05. A same-family mean control recovers 0.12 on its 41 eligible items. Permuting positions also gives 0.05, norm-matched Gaussian noise gives 0.02, and the identity control gives 0.00. These controls show that recovery depends on the injected content and its arrangement across positions, rather than perturbation size alone.

The receiver screen affects how these controls should be interpreted. On the 34 excluded calculators, the wrong-skill receiver already solves some items. Other-skill content recovers 0.17 on these groups ($n = 332$), compared with 0.05 on the retained groups ($n = 135$). Noise recovers 0.10 versus 0.02. This pattern is consistent with perturbations enabling knowledge already available to the model, although it does not establish that mechanism. A separate caveat is intervention size: skill spans in the screened subset contain 365–1,630 positions, far more than a single-position probe. §4.4 separates the number of written positions from their arrangement.

4.4 Which positions carry the effect

Span injection writes 365–1,630 positions, whereas the final-position probe writes one. We compare position selections at fixed written-token budgets B, using the same 135 items, layer and receiver as Table 2 (Figure 2). *Quantity alone is insufficient.* Evenly spaced or random subsets recover at most 0.15, even at half the span. Choosing the positions with the largest content change recovers at most 0.12. Writing after the span stays at or below 0.07, at layer 8 or the seven tested layers from 4 to 30. *Block selections differ.* Selecting 256 positions as one block recovers 0.37. Selecting them as 2, 4, 8, 16 and 32 spaced blocks gives 0.24, 0.22, 0.13, 0.08 and 0.08. The one-versus-32 difference is 0.29 [0.09, 0.48], with calculator-bootstrap uncertainty. These selections have similar mean energy shares (0.291 versus 0.295), but energy is not matched per item and the selected content changes: procedure coverage is 38.6% versus 30.7%. This does not isolate contiguity from semantic coverage. *Section and placement matter.* A half-span block centred on the procedure recovers 0.82 [0.67, 0.93], versus 0.18 around the example. Shifting each content difference by one position gives 0.15–0.22 with or without wrap-around. Omitting the first or last 16 positions but leaving the rest in place preserves 0.89–0.92. Recovery is thus sensitive to which content is injected and where, beyond the number of written positions.

4.5 Skill identity and content within the span

Table 3 examines which content supports transfer and whether it depends on the question. Of four contiguous span quarters, the formula quarter gives the largest standalone recovery (0.56), compared with 0.89 for the full span. With the skill before the question, causal masking prevents its states from depending on later question tokens: states are identical across instances at equal prompt length and nearly identical otherwise (cosine ≥ 0.994 over all 48 calculators). Placing the skill after the question allows such dependence. Even then, states from other instances of the same skill recover 0.98 at layer 8, close to the current instance's 1.02. At layer 16, the gap widens to 0.29 versus 0.49, while the instance-specific remainder recovers little on its own. Early transfer therefore relies largely on content shared across questions, with greater question dependence later.

Two completed corrected controls test whether a shared shift or a change in state norm could explain recovery. Averaging content differences across skills recovers only 0.03 with the skill first ($n = 135$) and 0.04 with it last ($n = 151$). Rescaling patched states to match the receiver's norm at each position preserves 0.87 recovery, compared with 0.89 without rescaling. In the corrected rerun, layer-16 recovery is 0.49 with the skill last (151 items, 17 calculators) and 0.06 with it first (135 items, 14 calculators). Because these screens retain different subsets, the contrast suggests sensitivity to prompt order but is not a matched estimate of its effect (extended version).

4.6 Rank required to recover the skill's effect

We next inject the truncated matrix in Eq. 4 to measure how recovery depends on the retained directions. Each intervention preserves the same skill-specific mean at every span position, so the comparison tests how much variation across positions is needed beyond that mean. At layer 8 on the screened MedCalc subset (135 items, 14 calculators), recovery is 0.07, 0.05, 0.12 for $k = 4, 16, 32$, rising to 0.63 at 64 and 0.77 at 128, and reaching the untruncated 0.89 at 256 (Figure 3). Keeping the bottom 4 to 128 directions recovers at most 0.04. Recovery therefore depends on which directions are retained: the leading directions support most of the measured effect, but a small number is insufficient under this intervention.

For comparison across models, we summarise each curve by k^*: the first rank reaching halfway from recovery at $k = 16$ to the maximum on the fixed grid $\{16, 32, 64, 128\}$, with interpolation in $\log_2 k$. We call this the *rank knee*, a summary of the measured recovery range. Qwen3-8B gives $k^* = 47$ [43, 55], approximately

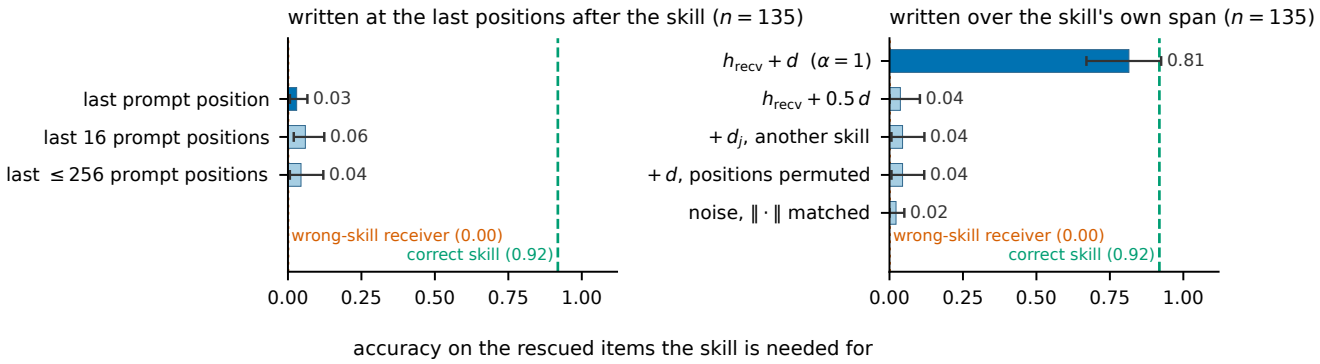


Figure 1: Matched position comparison on MedCalc with Qwen3-8B after the fix. Both panels use layer 8, the same wrong-skill receiver and 135 items from 14 calculators. Left: donor states written at the last 1, 16, or up to 256 available positions after the skill. Right: interventions over the skill span. The dashed line marks correct-skill accuracy; the dotted line marks receiver accuracy. Error bars are calculator-bootstrap 95% intervals.

Table 2: Span interventions on the screened MedCalc subset: 135 items from 14 calculators, selected from 467 rescued items across 48 calculators by requiring the wrong-skill receiver to solve no sampled item in a group. The same-family control is available on 41 items. Each row uses baselines matched to its own items in Eq. 3; accuracy intervals resample calculators. Table 4 gives the interval for full-span recovery.

intervention	n	accuracy	CI_{95}	recovery ρ
correct skill in context	135	0.919	[0.844, 0.976]	+1.00
$h_{recv} + d$ (full span)	135	0.815	[0.670, 0.925]	+0.89
correct content, shared prefix	135	0.593	[0.336, 0.820]	+0.65
half dose ($\alpha = 0.5$)	135	0.037	[0.000, 0.103]	+0.04
another skill (individual donor)	135	0.044	[0.007, 0.119]	+0.05
mean from another skill family	135	0.044	[0.012, 0.102]	+0.05
mean from the same skill family	41	0.122	[0.000, 0.250]	+0.12
positions permuted	135	0.044	[0.007, 0.118]	+0.05
norm-matched Gaussian noise	135	0.022	[0.000, 0.051]	+0.02
identity intervention	135	0.000	[0.000, 0.000]	+0.00
wrong skill, unpatched	135	0.000	[0.000, 0.000]	+0.00

1.1% of its hidden width. The extended version reports development and deeper-layer curves. These measurements describe what the injected representation can recover, rather than directly measuring the model's unmodified computation: truncated states may activate pathways absent from an unpatched run [23].

4.7 Transfer depth and continued use of the skill

Span replacement and attention knockout separate in depth. Figure 4 compares transfer through span replacement with continued use through attention on Qwen3-8B. Every layer is measured, and both interventions are read on the same screened items. Replacement recovers 0.92–0.98 through layer 7, 0.89 at 8 and 0.74 at 12 of 36, then 0.16 at 13 and at most 0.07 from 14 onward ($n = 135$, 14 calculators). The drop between layers 12 and 13 accounts for most of the decline. We define the crossing below $\rho = 0.5$ as the *transfer boundary* (labelled "handover" in the figures and tables). The log-probability of the donor's generated reasoning also drops between these layers, suggesting that binary answer grading sharpens, but does not fully explain, the transition (Appendix B). To test continued reading, *cumulative attention knockout* blocks attention to skill tokens at every layer from L onward. On the 134 rescued items of the same subset, 0.03–0.12 stay correct for $L \leq 23$. This rises to 0.25 at 24, first passes half at 25 (0.58) and reaches 0.94 at 35. The transfer boundary therefore lies at $13/36 = 0.36$ of depth and the reading boundary at $25/36 = 0.69$, measured on the same items under the same screen. Writing the last prompt position instead recovers at most 0.06 at any layer. Continued dependence on skill attention thus outlasts successful transfer from the skill's states. Transfer measures recovery in a wrong-skill receiver, whereas knockout measures dependence in a correct-skill run. The separation leaves open how span states interact with the receiver's other states, a question motivated by the multi-layer interactions found by Bigoulaeva et al. [3].

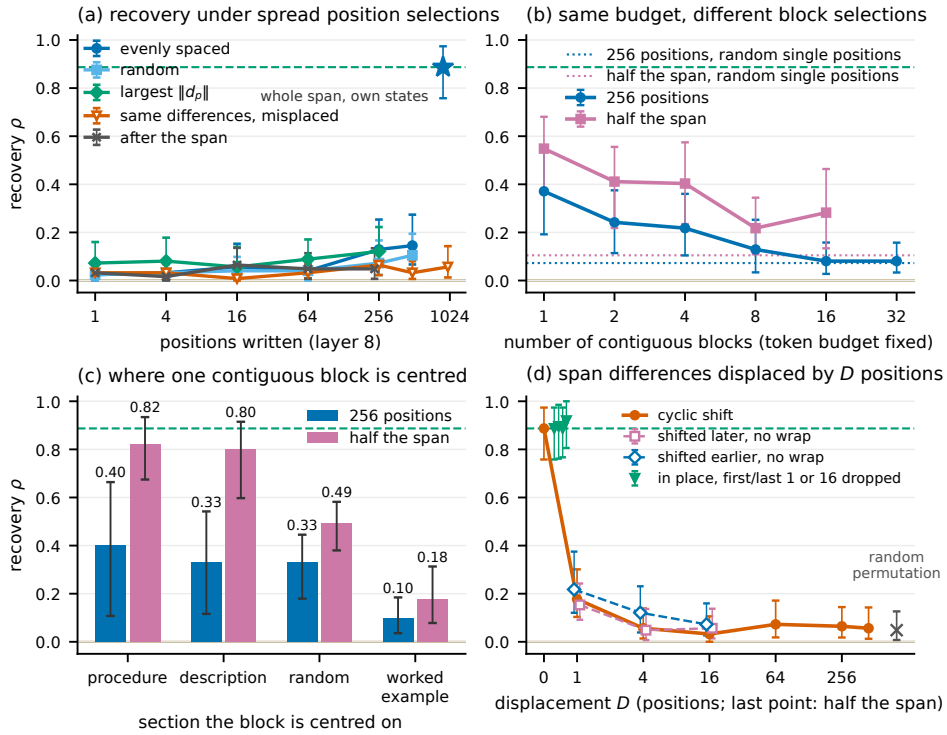


Figure 2: Position budget on MedCalc with Qwen3-8B, layer 8: the same 135 screened items (14 calculators) and wrong-skill receiver in every panel; the dashed line is the whole span written in place ($\rho = 0.89$). (a) B positions, each written with its own donor state (evenly spaced, random, or largest $\|d_p\|$), the corresponding differences written half a span away, or the last B prompt positions after the span. (b) A fixed budget written as N contiguous blocks; dotted lines are random single positions at the same budget. (c) One contiguous block centred on a section of the skill. (d) The span differences displaced by D positions, cyclically or without wrap-around; triangles write it in place without its first or last 1 or 16 positions. Error bars are calculator-bootstrap 95% intervals.

Adding late intervention layers does not restore transfer. To test whether a single late injection is simply too weak, we replace span states repeatedly within consecutive layer windows, using the battery's receiver and the same 135 items as the depth sweep. Windows 8–11, 12–15, 14–19 and 16–35 recover 0.89, 0.74, 0.16 and 0.06. The last equals layer 16 alone. Writing every layer at once recovers 0.98 [0.91, 1.03], so the transplant itself remains effective when it starts early enough. The identity control is exact on all 467 items. On TheoremQA (99 items, 75 groups, with its battery's receiver), the same windows recover 0.84, 0.61, 0.38, 0.20, the last again equal to layer 16 alone. Repeated late injection therefore does not restore early-layer recovery in these tests.

A spectral drop is not evidence of compression. Could the loss of transfer reflect compression into fewer dimensions? We examine the centred participation ratio, a summary of how broadly matrix energy is distributed across singular directions. It falls from 68 at layer 12 to 1.0 at layer 16, but this drop is driven by unusually large activations [30, 34]. In 35 of 48 calculators, one token's state reaches 121–195 times the median span norm, always on the same two hidden dimensions. Removing the four highest-norm positions restores the ratio to 85 at layer 16, and it stays between 62 and 98 over layers 0–28. The drop is also absent in Qwen3-0.6B and Qwen3-1.7B. This diagnostic therefore does not support compression as the explanation for lost transfer: a few large activations distort the dimensionality summary (extended version).

4.8 Replication across models and task domains

We extend span injection, rank truncation and the available depth comparisons to Qwen3-0.6B ($D = 1024$), Mistral-7B ($D = 4096$, 32 layers), and TheoremQA. Each model is injected at an early layer, and the receiver screen is applied separately to each run. Table 4 distinguishes completed comparisons from missing controls or attention knockouts. Results for Qwen3-1.7B, where skills rescue only 8–11% of items, appear in the extended version.

Span recovery ranges from 0.74 to 0.94 across the six model–task combinations. The largest control is 0.35, for a same-family mean on 31 Mistral/MedCalc items, where full-span recovery on the same items is 0.71. In every model–task combination, the transfer boundary occurs before the reading boundary. The measured transfer boundaries lie at 0.33–0.43 of model depth within Qwen and at 0.19–0.22 in Mistral. Reading boundaries also vary by task:

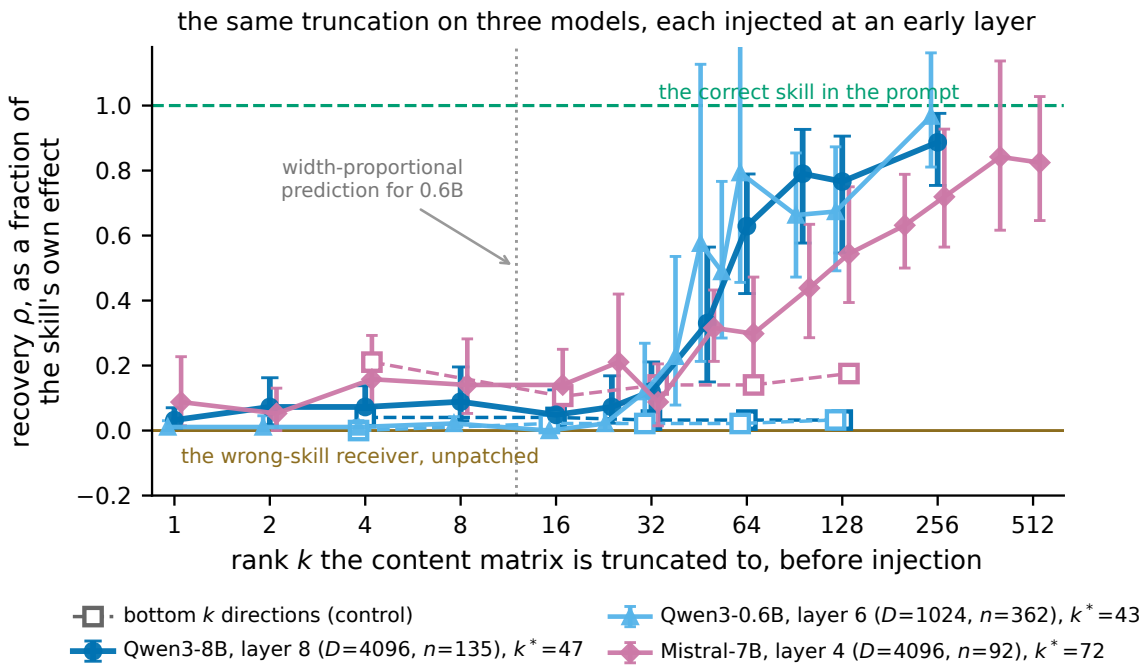


Figure 3: Recovery after retaining k leading centred directions and restoring the positional mean, on MedCalc after the fix. Each model is injected at an early layer. Despite a fourfold difference in hidden width, Qwen3-0.6B and Qwen3-8B have similar rank knees (43 and 47); Mistral-7B, with the same width as Qwen3-8B, has $k^* = 72$ [46, 94]. Untruncated recovery is 0.89, 0.91 and 0.84, respectively, on each model's screened subset (135, 362 and 92 items). Both Qwen models reach that value by $k = 256$ and Mistral-7B at $k = 384$. Dashed curves retain bottom directions. Error bars are calculator-bootstrap 95% intervals. Only complete rank experiments are plotted.

Table 3: Content and question dependence of span states, Qwen3-8B after the fix. Top: each contiguous quarter is injected separately, on 135 items from 14 calculators. Bottom: with the skill after the question, content shared across other instances and the instance-specific remainder are injected separately, on 151 items from 17 calculators. Each comparison uses its screened subset from the larger rerun. All entries report recovery ρ.

Skill quarters, layer 8, skill before question ($n = 135$)	
Injected content	Recovery ρ
Prose description	0.09
Formulas	0.56
Tool signatures	0.12
Worked example	0.05
Full span	0.89

Question dependence, skill after question ($n = 151$)		
Injected content	Layer 8	Layer 16
This instance	1.02	0.49
Other instances of the same skill	0.98	0.29
Instance-specific remainder	0.03	0.05

0.58–0.71 on TheoremQA and 0.69–0.84 on MedCalc across the three models.

The pre-registered width-proportional prediction places Qwen3-0.6B's rank knee near 12. Its observed recovery at $k = 16, 32, 64, 128$ is 0.00, 0.12, 0.79, 0.67, giving $k^* = 43$ [39, 52], close to Qwen3-8B's 47 [43, 55] despite the fourfold width difference. On TheoremQA, the corresponding knees are 52 [46, 62] and 50 [39, 67], again inconsistent with proportional scaling. Their participation ratios are also similar in scale (65–75 versus 70–99 over layers 0–20, excluding unusually large activations). Mistral, with the same width as Qwen3-8B, gives $k^* = 72$ [46, 94] on MedCalc and 64 [50, 79] on TheoremQA: higher than Qwen3-8B on both tasks, but with overlapping intervals. Its curve is also less sharply defined, since recovery keeps rising past $k = 128$ (0.54; 0.72 at 256) and reaches its untruncated 0.84 at $k = 384$, against 256 for Qwen3-8B. Its bottom-k controls reach 0.11–0.21. The measured rank requirement is therefore not determined by hidden width alone, while a difference between families is not established.

Readout checks. Alternative readouts can rank interventions differently. On the synthetic task, averaged gold-answer log-probability favours a mean shared across skills by 5.05 nats despite lower accuracy (0.251 versus 0.288, 358 items). Forcing an `ANSWER:` cue on MedCalc cuts the skill's accuracy gain from 39.7 to 4.7

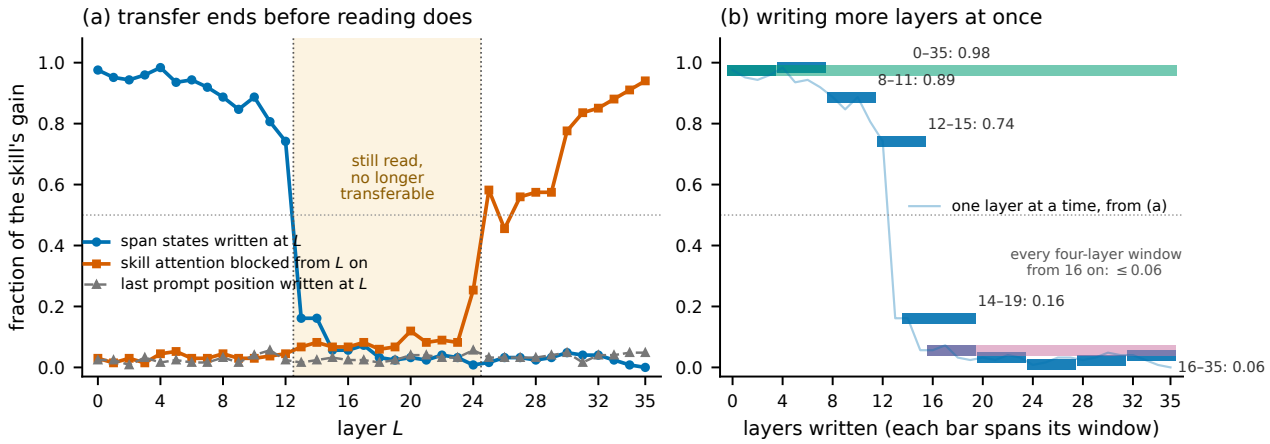


Figure 4: Transfer declines earlier than dependence on skill attention, Qwen3-8B on MedCalc, with every layer measured. (a) Three interventions on one axis and the same screened items (n = 135; the knockout on their 134 rescued items): span states written at layer L; attention to the skill blocked from L onward; and the last prompt position written at L. The shaded band spans the layers where skill attention still supports performance but span replacement recovers less than half of the gain. (b) Each window is drawn across the layers it writes: four-layer windows tile the stack, and the long windows 14–19, 16–35 and 0–35 are shown with them; the faint curve repeats (a). Writing every layer at once recovers 0.98, while any window starting at 16 or later stays at or below 0.06. Span interventions use the corrected implementation; attention knockout does not use state capture.

Table 4: Replication with the corrected implementation. Injection layers are 8 for Qwen3-8B, 6 for Qwen3-0.6B and 4 for Mistral-7B. Recovery intervals resample skill groups. Max control includes half-dose, other-skill, permutation and noise comparisons, including same-family means available on 41 MedCalc/Qwen3-8B and 31 MedCalc/Mistral items. n describes the battery. Handover brackets the transfer boundary as a fraction of model depth; reading is the first layer, at one-layer resolution, from which blocking attention to the skill still leaves at least half of the rescued items correct, measured on the same screened items as the transfer boundary. k^* is the rank knee on the fixed grid.

task	model	n (groups)	ρ [95% CI]	max control	handover	reading	k^*
MedCalc	Qwen3-8B	135 (14)	0.89 [0.76, 0.97]	0.12	0.33–0.36	0.69	47
TheoremQA	Qwen3-8B	99 (75)	0.84 [0.75, 0.92]	0.25	0.33–0.36	0.58	50
MedCalc	Qwen3-0.6B	362 (38)	0.91 [0.75, 1.05]	0.30	0.36–0.39	0.79	43
TheoremQA	Qwen3-0.6B	132 (97)	0.74 [0.66, 0.82]	0.13	0.36–0.39	0.71	52
MedCalc	Mistral-7B	92 (20)	0.84 [0.69, 1.09]	0.35	0.19–0.22	0.84	72
TheoremQA	Mistral-7B	115 (94)	0.94 [0.84, 1.05]	0.27	0.19–0.22	0.59	64

points (0.178 versus 0.131 on 720 items) while leaving a +1.08 nat log-probability difference. Synthetic multiple choice retains 0.54 of the skill's effect after its numbers are corrupted and 0.82 after its lines are scrambled, illustrating possible answer-format shortcuts [2] (extended version). Other domains give limited evidence (extended version).

5 CONCLUSION

In the evaluated tasks and screened subsets, states across a skill's own tokens transfer much of its effect, while a final-position intervention is more sensitive to answer format in our one-time patching tests. Span recovery depends on content, position and multiple leading directions. The rank knee does not scale proportionally with width within Qwen, and a difference between model families is suggested but not established. Across layers, states stop transferring most of the effect before attention to them ceases to support performance, and the spectral drop does not establish compression at that boundary. These findings distinguish what skill states can transfer from continued dependence on reading them. Explaining how the receiving computation constrains transfer is a next step.

AI USE STATEMENT

Generative AI tools assisted with manuscript restructuring and language editing, literature lookup and reference checks, consistency checks between reported results and local experiment records, and figure-label editing. The authors are responsible for the final text, scientific claims and experimental artifacts.

REFERENCES

[1] Thibaud Ardoin, Semira Einsele, Evis Bregu, and Gerhard Wunder. 2026. Prompt Compression via Activation Aggregation. *arXiv preprint arXiv:2607.08399* (2026).

[2] Nishant Balepur, Abhilasha Ravichander, and Rachel Rudinger. 2024. Artifacts or Abduction: How Do LLMs Answer Multiple-Choice Questions Without the Question?. In *Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:2402.12483.

[3] Irina Bigoulaeva, Jonas Rohweder, Subhabrata Dutta, and Iryna Gurevych. 2026. Patches of Nonlinearity: Instruction Vectors in Large Language Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 12209–12262. https://doi.org/10.18653/v1/2026.acl-long.559

[4] Lucas Caccia, Alan Ansell, Edoardo Ponti, Ivan Vulić, and Alessandro Sordoni. 2025. Training Plug-n-Play Knowledge Modules with Deep Context Distillation. In *Conference on Language Modeling (COLM)*. arXiv:2503.08727.

[5] Rujikorn Charakorn, Edoardo Cetin, Shinnosuke Uesaka, and Robert Tjarko Lange. 2026. Doc-to-LoRA: Learning to Instantly Internalize Contexts. *arXiv preprint arXiv:2602.15902* (2026).

[6] Guy Davidson, Todd M. Gureckis, Brenden M. Lake, and Adina Williams. 2025. Do different prompting methods yield a common task representation in language models? *arXiv preprint arXiv:2505.12075* (2025). https://arxiv.org/abs/2505.12075

[7] Yuxin Dong, Jiachen Jiang, Zhihui Zhu, and Xia Ning. 2026. Understanding Task Vectors in In-Context Learning: Emergence, Functionality, and Limitations. In *International Conference on Learning Representations (ICLR)*. arXiv:2506.09048.

[8] Murat Dura, Serkan Öztürk, and Selma Tekir. 2026. Mechanistic Interpretability of Chain-of-Thought Reasoning via Sequential Activation Patching. *arXiv preprint arXiv:2608.22332* (2026).

[9] Jesseba Fernando and Grigori Guitchounts. 2026. Dynamics of the Transformer Residual Stream: Coupling Spectral Geometry to Network Topology. *arXiv preprint arXiv:2605.14258* (2026).

[10] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. In-context Autoencoder for Context Compression in a Large Language Model. In *International Conference on Learning Representations (ICLR)*. arXiv:2307.06945.

[11] Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. 2023. Dissecting Recall of Factual Associations in Auto-Regressive Language Models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. arXiv:2304.14767.

[12] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? *arXiv preprint arXiv:2603.15401* (2026).

[13] Horace He and Thinking Machines Lab. 2025. Defeating Nondeterminism in LLM Inference. Thinking Machines Lab: Connectionism. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

[14] Stefan Heimersheim and Neel Nanda. 2024. How to use and interpret activation patching. *arXiv preprint arXiv:2404.15255* (2024).

[15] Roee Hendel, Mor Geva, and Amir Globerson. 2023. In-Context Learning Creates Task Vectors. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. arXiv:2310.15916.

[16] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. 2023. Editing Models with Task Arithmetic. In *International Conference on Learning Representations (ICLR)*. arXiv:2212.04089.

[17] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).

[18] Nikhil Khandekar, Qiao Jin, Guangzhi Xiong, Soren Dunn, Serina S. Applebaum, Zain Anwar, Maame Sarfo-Gyamfi, Conrad W. Safranek, Abid A. Anwar, Andrew Zhang, Aidan Gilson, Maxwell B. Singer, Amisha Dave, Andrew Taylor, Aidong Zhang, Qingyu Chen, and Zhiyong Lu. 2024. MedCalc-Bench: Evaluating Large Language Models for Medical Calculations. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. arXiv:2406.12036.

[19] Olga Kovaleva, Saurabh Kulshreshtha, Anna Rogers, and Anna Rumshisky. 2021. BERT Busters: Outlier Dimensions that Disrupt Transformers. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. arXiv:2105.06990.

[20] Xiangyi Li, Yimin Liu, Wenbo Chen, et al. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670* (2026).

[21] Yuxuan Li, Declan Campbell, Stephanie C. Y. Chan, and Andrew Kyle Lampinen. 2025. Just-in-time and distributed task representations in language models. *arXiv preprint arXiv:2509.04466* (2025).

[22] Sheng Liu, Haotian Ye, Lei Xing, and James Y. Zou. 2024. In-context Vectors: Making In Context Learning More Effective and Controllable Through Latent Space Steering. In *International Conference on Machine Learning (ICML)*. arXiv:2311.06668.

[23] Aleksandar Makelov, Georg Lange, and Neel Nanda. 2024. Is This the Subspace You Are Looking for? An Interpretability Illusion for Subspace Activation Patching. In *International Conference on Learning Representations (ICLR)*. arXiv:2311.17030.

[24] Houman Mehrafarin, Amit Parekh, and Ioannis Konstas. 2026. When Chain-of-Thought Fails, the Solution Hides in the Hidden States. In *Findings of the Association for Computational Linguistics: EMNLP 2026*. arXiv:2604.23351.

[25] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and Editing Factual Associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2202.05262.

[26] Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2023. Learning to Compress Prompts with Gist Tokens. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2304.08467.

[27] Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. 2024. Steering Llama 2 via Contrastive Activation Addition. In *Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:2312.06681.

[28] Aditya Pola and Vineeth N. Balasubramanian. 2025. Where does an LLM begin computing an instruction?. In *UniReps Workshop*. extended abstract; arXiv:2511.10694.

[29] Nilesh Sarkar and Dawar Jyoti Deka. 2026. Causal Dimensionality of Transformer Representations: Measurement, Scaling, and Layer Structure. *arXiv preprint arXiv:2605.08740* (2026).

[30] Zeru Shi, Zhenting Wang, Fan Yang, Qifan Wang, and Ruixiang Tang. 2026. A Single Layer to Explain Them All: Understanding Massive Activations in Large Language Models. *arXiv preprint arXiv:2605.08504* (2026).

[31] Suzanna Sia, David Mueller, and Kevin Duh. 2024. Where does In-context Learning Happen in Large Language Models?. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2403.04510.

[32] Alessandro Stolfo, Vidhisha Balachandran, Safoora Yousefi, Eric Horvitz, and Besmira Nushi. 2025. Improving Instruction-Following in Language Models through Activation Steering. In *International Conference on Learning Representations (ICLR)*. arXiv:2410.12877.

[33] Weihang Su, Jianming Long, Qingyao Ai, Qiaozhi He, Yichen Tang, Changyue Wang, Yiteng Tu, Yingbo Wang, and Yiqun Liu. 2026. Skill Retrieval Augmentation for Agentic AI. *arXiv preprint arXiv:2604.24594* (2026).

[34] Mingjie Sun, Xinlei Chen, J. Zico Kolter, and Zhuang Liu. 2024. Massive Activations in Large Language Models. In *Conference on Language Modeling (COLM)*. arXiv:2402.17762.

[35] William Timkey and Marten van Schijndel. 2021. All Bark and No Bite: Rogue Dimensions in Transformer Language Models Obscure Representational Quality. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. arXiv:2109.04404.

[36] Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. 2024. Function Vectors in Large Language Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.15213.

[37] Lucrezia Valeriani, Diego Doimo, Francesca Cuturello, Alessandro Laio, Alessio Ansuini, and Alberto Cazzaniga. 2023. The Geometry of Hidden Representations of Large Transformer Models. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2302.00294.

[38] Sarah Wiegreffe, Oyvind Tafjord, Yonatan Belinkov, Hannaneh Hajishirzi, and Ashish Sabharwal. 2025. Answer, Assemble, Ace: Understanding How LMs Answer Multiple Choice Questions. In *International Conference on Learning Representations (ICLR)*. arXiv:2407.15018, spotlight.

[39] An Yang, Anfeng Li, et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).

[40] Fred Zhang and Neel Nanda. 2024. Towards Best Practices of Activation Patching in Language Models: Metrics and Methods. In *International Conference on Learning Representations (ICLR)*. arXiv:2309.16042.

A SETUP, IN FULL

Two task families. Real skills, the main material: MedCalc-Bench instances with the 55 gold calculator skills released with SRA-Bench—55 calculators × 20 instances = 1,100 items, graded deterministically against a numeric tolerance with no model in the loop. Both the skills and the tasks are other people's, which is what makes the effect size credible. Qwen3-8B, single turn, no tools, greedy. Behavioural baseline and replication intervals resample skill groups because items sharing a skill are dependent; original item-level intervals in individual tables and plots are identified separately in the main text. *A synthetic positive control*: a fictional unit system with a one-page document giving its conversion table, so the no-document floor is guaranteed, contamination is impossible, and we know exactly what content means. Items are

generated from the table ($n = 358$), and every item whose answer appears in a worked example is dropped.

Two dependent variables. Accuracy, from a greedily decoded free-form answer graded by the task's own scorer, and the teacher-forced log-probability of the gold answer. They fail differently and disagree often enough that reporting one alone is unsafe (§4.8); accuracy is read first throughout. No item was rewritten into multiple choice to obtain an accuracy channel—The extended version shows why that rewrite is not harmless.

How to read an arm's number. Interventions are reported both as raw accuracy and as the fraction of the range an intervention could possibly recover,

$$\rho(\text{arm}) = \frac{\text{acc}(\text{arm}) - \text{acc}(\text{receiver})}{\text{acc}(\text{donor}) - \text{acc}(\text{receiver})}, \tag{5}$$

where the receiver is the prompt being patched into, unpatched, and the donor is the document actually in the prompt. ρ is what transfers across settings and raw accuracy is not: the four configurations in the answer-length ladder have donors at 0.550, 0.467, 1.000 and 0.950 and receivers from 0.000 to 0.350, and an earlier draft of this paper reported a channel as *stronger* under a longer answer by comparing 0.408 against 1.000 when as fractions they are 0.87 and 1.00. Every figure axis labelled "fraction" is Eq. 3.

The instrument's own noise floor, measured. Every decoding run carries a *null* arm: the receiver's own span states are captured and written back over themselves. With a length-matched receiver this is a bit-for-bit identity, so the null arm must equal the unpatched receiver at every layer. Before the fix of Appendix A.1 it did not, quite. On Qwen3-8B, across 1,766 paired observations from every pre-fix run that carries both arms, the null arm agrees with the receiver on 93.9% of items: 108 flip, 46 from wrong to right and 62 from right to wrong, symmetric within a sign test ($p = 0.15$). The pipeline itself is deterministic—the same configuration run on two different machines agreed on all 360 item-by-arm cells—so this is not run-to-run variation but a reproducible property of the instrument: the capture and decoding passes took different attention kernels, which perturbs the written-back state by 1.2% at layer 8, and the dependent variable is the end of a chain of thought several hundred tokens long, which is long enough for that to change an argmax somewhere and propagate. It scales with the model: the same arm flips 0.6% of items on Qwen3-0.6B (1,696 pairs) and 0.8% on Qwen3-1.7B (1,338), against 6.1% on the 8B. After the fix it is exact (80/80).

Three consequences are used throughout. First, **an effect smaller than about six points is not resolvable on the 8B** in pre-fix runs, whatever a binomial interval says. Second, the floor is not uniform: on the persistent-failure cell it is 16%, between two and three times the average, which is what one expects of items on a decision boundary, so numbers read on that cell are read against a coarser ruler. Third, on the restricted split the pre-fix null arm read 0.034 rather than zero—the split conditions on the receiver failing, so a flip there can only go upward—so pre-fix span numbers are read against 0.034; after the fix it reads exactly 0.000, the floor Table 2's 0.750 is read against. We have rarely seen this control reported in the patching literature; at a single answer token it is small enough to be invisible, and it becomes large only once the dependent variable is a long generation.

Four cells. Every item is labelled by what the document did to it: **R** (rescued: wrong without, right with), **F** (persistent: wrong both ways), **K** (kept: right both ways), **B** (broken: right without, wrong with). **R is the only cell in which the document demonstrably works**, every table here is read by cell, and §4.8 shows the aggregate over all four can be sign-inverted relative to R.

A.1 Experimental detail

Models and numerics. Qwen3-1.7B (28 layers, $d = 2048$) and Qwen3-8B (36 layers, $d = 4096$); §4.8 adds Qwen3-0.6B (28 layers, $d = 1024$) and Mistral-7B-Instruct-v0.3 (32 layers, $d = 4096$), whose chat template has no system role, so the system text is merged into the user turn. Greedy decoding throughout, `enable_thinking` disabled and verified by a self-test on the rendered template. Eager attention wherever an attention mask is manipulated, because sdpa and flash silently ignore a 4D mask; layer-output hooks are implementation-independent, so the decode-heavy runs use sdpa, which is about 2.5× faster on a 2k-token prompt. The 1.7B synthetic runs are fp32: bf16 costs 0.5–2.5 nats on the log-probability of a token deep in the tail, and every log-probability quantity here is a difference of such numbers. The 8B runs are bf16 and the gaps we report from them are 5–8 nats, well outside that error; the accuracy channel is immune to it either way.

Self-tests, and one that failed. Three checks run with the experiments. (i) *Self-transplant*: donor and receiver are the same forward, so the patch must be an exact no-op. On the fp32 synthetic runs the deviation is 0.0 nats at every layer. On the bf16 8B runs it is up to 0.38 nats—bf16 arithmetic over two sequence lengths—while the accuracy channel remains an exact no-op at every layer, which is how we tell the two apart. (ii) *Last-layer patch*: replacing the final layer's output at the patched position makes that position's logits the donor's by construction, so the next token must match exactly. (iii) *Query-side control* for the knockouts.

An error worth recording. The first version of the real-skill scoring code patched at index −1 of a forward run over prompt *and* answer, which is the last token of the answer—after every position whose logits are scored. Every arm returned the untouched baseline to six decimals, which is what made it visible. Relative indices are the wrong default whenever the forward length varies between the capture and the patch.

Control documents. The synthetic controls are generated from the skill document by `make_controls.py`, which checks two things: the corrupted document's numbers must not coincide with the task set's gold answers (7 of 249 do, 2.8%), and the shuffle must not be the identity. All three controls are stored under the same filename in separate directories, because the injector derives the "# Skill: *name*" header from the file stem—a file called `SKILL.zorb-units.corrupted.md` would announce the manipulation in the prompt, and the control would then differ from the treatment in two ways.

Item set. The synthetic set is generated from the conversion table with the value range 2–40, dropping any item whose answer

also appears in a worked example of the document ($n = 358$; the sweeps use the first 120 or 200). Distractors are values reachable by misreading the table—the neighbouring row, the wrong family, the inverted direction—never random numbers, and the key is rotated across A/B/C/D so a position-biased model still scores at chance.

A.2 Evaluation scope and intervention provenance

An accidental replication. A queueing mistake ran one 36-layer synthetic sweep (358 items, two arms, generations up to 900 tokens) twice, on two different GPUs in two different containers. All 36 layer files agree byte for byte. This does not bound the nondeterminism discussed in §4.1—it is one configuration on one pair of devices—but it is the only end-to-end check of it we have, and it came out exact.

Sample size and scope. The battery was developed on $n = 40$ (ten calculators × four rescued instances), which gives intervals of roughly ±0.15; §4.3 reruns it on forty calculators and §4.8 on further models and a second task, and the conclusions are the ones that survived that. We report profiles and not individual layers: the one non-monotonicity in the depth curve (0.250 at layer 18 against 0.475 at 20) has overlapping intervals and we do not interpret it. Among the three models, Qwen3-1.7B is the weakest rung: the document rescues only 8–11% of its items, so its truncation curve is not resolved and is reported for completeness rather than relied on. The width argument rests on Qwen3-0.6B against Qwen3-8B, a factor of four within one family; a second family of the same width needs more directions (extended version). In the pre-fix development runs, four independent shards recomputed the three baselines and agreed to the item (0.950 / 0.025 / 0.175), and the no-op arm sits on the receiver's baseline at all twelve layers to within the 5% numerical floor of §4.1, so the instrument is checked even where the estimate is coarse. The span channel, the depth sweep and the truncation curve have three models and two tasks; the knockouts two models and two tasks; the final-position results two models (Qwen3-1.7B, fp32); the document-order comparison and the within-document localisation one.

Which items the causal claims are about. Two restrictions stack. Causal claims are made only on the rescued cell, the only cell in which the document demonstrably works; on the persistent-failure cell the same arms track the document rather than correctness—the transplant fails exactly where the document fails—which is the right behaviour but is not a substitute for the other cells. And within that cell the span arms are read on the calculators whose *receiver* never solves an item, because on the others the model does not need the document, and before the fix any disruption of the wrong one, noise included, let it fall back on what it already knows ($\rho = 0.14$ for noise there against 0.00 here). After the fix noise no longer does that (0.00 on those calculators), but a donor from another skill still reads 0.47 there against 0.19 on the rest. On forty calculators the restriction keeps fifteen of them and 60 items. On this split, accuracy is 0.750 for the transplant and at most 0.083 for the tested negative controls; the main text distinguishes item-level intervals from group-bootstrap uncertainty. Those forty calculators are themselves the 160 rescued items with the most instances per calculator, out of 467. We therefore reran the battery, the truncation curve, the depth sweep, the dose curve and the within-document quarters over *every* rescued item of every calculator with at least two (467 items, 48 calculators; the restriction then keeps 135 items over 14). Every conclusion holds and no control moves above 0.12; Appendix B gives the comparison arm by arm.

What the rank result does and does not establish. §4.6 truncates the injected content matrix to rank k at a layer where the untruncated version recovers most of the effect, and recovery stays near the control floor at $k \leq 32$ while the bottom-k control stays at the floor. That licenses the forward direction: the effect requires directions of the same order as the span's participation ratio, and within the Qwen family neither number scales with width. It does not license a claim that the number belongs to the document alone (Mistral-7B needs more), nor a converse about depth. Supplying a deep layer with a higher-rank message taken from a shallow one does not restore the handover ($\rho = 0.00$ against 0.06 for the same-layer donor, 0.00 with the norm matched), so we do not claim that what a deep layer lacks is rank.

Diagnostic results for the behavioural readout. We could not find any representational quantity at the final position that distinguishes the items a skill rescues from the ones it does not: the norms and angles agree to three digits, and the one similarity measure that separates them under the gold skill separates them equally under a wrong one. And the document acts through a chain of thought, so every causal measurement decodes several hundred tokens; scoring under a forced `ANSWER:` cue instead leaves almost no behavioural effect (0.178 with the skill, 0.131 without and 0.100 with a wrong one, on 720 items) while retaining a +1.08 nat log-probability difference—so the span channel has no usable log-probability reading, and is reported on accuracy alone.

An instrument fault and result provenance. State capture omitted the attention mask supplied during decoding. Although mathematically equivalent for an unpadded sequence, these calls selected different kernels. Their states differed by 1.2% at layer 8 and 25% at layer 34, and identity patching changed 6.1% of Qwen3-8B answers. Matching the masks restores exact identity agreement. All main-text state interventions use the corrected implementation, including the synthetic and final-position sweeps (§4.1). Attention knockout does not use state capture. Historical development results in the extended appendices retain their original sample sizes and provenance. The corrected full-data rerun in Appendix B supersedes the earlier 60-item battery.

Numerics. Log-probabilities on the synthetic control are fp32, because bf16 costs 0.5–2.5 nats on a token deep in the tail. The 8B numbers are bf16, which appears as self-transplant deviations of 0.1–1.1 nats while the accuracy channel remains an exact no-op; accuracy is read first throughout.

B THE FULL-DATASET RERUN, AND WHAT READS THE SPAN

Every MedCalc × Qwen3-8B result in the main text was developed on the forty calculators with the most rescued items, sampling four

Table 5: Full-dataset rerun against the subset the main text cites, Qwen3-8B on MedCalc, layer 8. Recovery is Eq. 3; intervals are group bootstrap over calculators. *per-calc* is the main text's restriction, *per-item* the weaker one. Controls reach 0.12 under the group screen and 0.25 under the item screen; each arm uses its own matched baselines.

arm	full, per-calc (n = 135)	subset (n = 60)	full, per-item (n = 369)
transplant `real`	**0.89** [0.76, 0.97]	0.87	0.94
same skill, matched prefix	0.65 [0.37, 0.87]	0.73	0.84
half dose $\alpha = 0.5$	0.04	0.06	0.11
positions permuted	0.05	0.08	0.21
norm-matched noise	0.02	0.10	0.18
other calculator	0.05	—	0.25
other family, mean	0.05	0.06	0.16
same family, mean	0.12 [0.00, 0.25]	0.05	0.24
identity patch	0.00 [0.00, 0.00]	0.00	0.00
rank 128 / 64 / 32 / 16	0.77/0.63/0.12/0.05	0.77/0.54/0.13/0.06	0.88/0.70/0.22/0.18
bottom-64 / bottom-16	0.03/0.04	0.00/0.06	0.11/0.12
quarters $q_0/q_1/q_2/q_3$	0.09/**0.56**/0.12/0.05	0.12/0.56/0.12/0.12	0.18/0.64/0.22/0.19

each: 160 of 467 rescued items, and 60 items over 15 calculators after the receiver-never-solves restriction. We reran each of them with every rescued item of every calculator that has at least two (467 items, 48 calculators), changing nothing else. The restriction is stricter on the larger set—a calculator now has to fail on up to twenty items rather than four—so it keeps 135 items over 14 calculators, and we also report the per-item restriction (369 items over 45), which keeps the items whose own receiver fails.

The transplant, the truncation knee between 32 and 64 directions, and the formula quarter's dominance all reproduce. The one arm that moves is the same-family donor, from 0.05 to 0.12, on the 41 items where it is defined.

The depth sweep at one-layer resolution. The subset sweep measured layers 0, 4, 8, 12, 14, 15, 16, 20 and placed the collapse somewhere after 12. At full scale we added 13 and 17–19: recovery is 0.98, 0.98, 0.89, 0.74 at layers 0, 4, 8, 12, then 0.16, 0.16, 0.06, 0.06, 0.07, 0.03, 0.02, 0.03 at 13 through 20 (per-calc; the per-item restriction gives 1.00, 0.98, 0.94, 0.79 then 0.32, 0.34, 0.25, 0.25, 0.23, 0.19, 0.14, 0.17). Layers 13 and 14 are indistinguishable under both restrictions, so the transition is a single layer between 12 and 13 and not the gradual decline a stride-2 sweep suggests.

A continuous dependent variable at the same boundary. Accuracy over a chain of thought is a threshold, and a smooth decline inside the model could present as a step in it. We therefore decode the with-document trajectory once per item and teacher-force that same trajectory under every other condition, reading its mean token log-probability: $q = (\ell_L - \ell_{\text{recv}})/(\ell_{\text{gold}} - \ell_{\text{recv}})$ has the same two anchors as ρ with no decoding threshold anywhere in it (n = 442 items on which the document run is correct, 48 calculators). q reads 0.999, 0.989, 0.960, 0.831 at layers 0, 4, 8, 12, then 0.598 [0.550, 0.648] at 13, 0.595 at 14, 0.340 at 16, 0.201 at 20 and 0.129 at 24. Expressed per layer as a fraction of its range, the step at 12 → 13 is 26.8%, against 3.7% per layer over 8–12 and 0.3% from 13 to 14: the discontinuity is present in a readout that cannot be produced by grading. It is smaller than the behavioural step (61% of range), so the grader sharpens the boundary roughly twofold without creating it, and it is spread evenly across the four quarters of the trajectory (0.20, 0.26, 0.31, 0.21) rather than concentrated at the answer, which is the opposite of what a grading artefact predicts. We also tried restricting q to the tokens after the final answer marker and report that it carries no information here: once the full reasoning is teacher-forced, every condition predicts the final number almost perfectly (median ℓ between −0.001 and −0.005 for all of them, and the gold-minus-receiver denominator has median 0.0014, below 0.01 on 59% of items).

Which heads read the span. We recorded each head's contribution from the span to later positions, $W_O^h \sum_{s \in \text{span}} a_{t,s} v_s$, comparing the document and filler runs (n = 455). Raw magnitudes are dominated by the last layers, but the residual stream grows by more than an order of magnitude with depth; normalised by it, the read-out concentrates in layers 12–21 and peaks at 18. It is not a small circuit: of the 1,152 heads the top 1% carry 10.5% of the read-out and the top 5% carry 31.1%, and no head attends to the span more than uniformly (0.4–1.2×, with the span occupying half the prompt). Blocking the 32 highest-ranked heads' attention to the span—ranked on half the calculators and evaluated on the held-out half—costs 0.39 of the document's effect, against 0.23 for 32 random heads at the same layers (paired difference 0.154, 95% CI [0.061, 0.251], McNemar 39 versus 9 discordant, n = 195 over 23 calculators). The ranking is therefore causal, but it buys only 0.15 over chance and 32 heads remove well under half the effect: the span is read diffusely, and we do not claim a nameable set of reader heads. One further observation constrains what fails at depth. After an injection at layer L, the span's own states from $L + 1$ onward are identical to the document run by construction—the span attends only to a shared prefix and to itself—and the downstream per-head read-out of the span is indeed 0.86–0.99 of the document's at every injection layer we tried, including those at which behaviour is gone. This read-out is compatible with continued access to donor content. A late span patch leaves the receiver's earlier question-state updates unchanged, motivating a direct test of their interaction with the transplanted states.